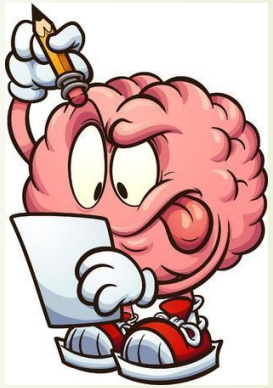
UT3_III. Otros diagramas de comportamiento

1. Introducción al UML Estructural

- Los diagramas UML estructurales representan la **estructura estática** de un sistema y sus **componentes**.
- Muestran la **jerarquía** de los **componentes** o módulos, y la **forma** en que se **conectan** o **interactúan** entre sí.
- Existen diferentes tipos de diagramas UML estructurales:
  - Diagramas de **clases**.
  - Diagramas de componentes.
  - Diagramas de despliegue.
  - …..

2. UML de Comportamiento

- El diagrama UML de **comportamiento** representa cómo **interactúan** y se **comportan** los elementos de un sistema. Estos diagramas modelan procesos, flujos de trabajo, interacciones y dinámicas.
- Incluyen los siguientes **tipos** principales:
  - Diagrama de **casos** de **uso**.
  - Diagrama de **actividades**.
  - Diagrama de **secuencia**.
  - Diagrama de **comunicación/colaboración**



- Los diagramas del tipo **casos de uso** y **actividades** se centran en el sistema desde un punto de vista más abstracto o funcional.
- Los diagramas del tipo **secuencia** y **comunicación/colaboración** son más específicos, modelando cómo los componentes **interactúan** y cambian en un contexto dado.

3. Diagrama UML de actividades. Notación

- Los **diagramas de actividades** en UML son una representación visual del **flujo de trabajo** o **procesos** dentro de un sistema. Se utilizan para modelar la **secuencia de actividades** en un proceso de negocio, un caso de uso o un algoritmo.
- Características principales:
  - Representan el flujo de control y de datos dentro de un proceso.
  - Modelan procesos de negocio y algoritmos de forma clara y estructurada.
  - Pueden incluir decisiones, bucles y concurrencia.
  - Son similares a los diagramas de flujo, pero con más elementos específicos de UML.
  - Se pueden aplicar a sistemas software, procesos organizacionales y automatización de flujos de trabajo.

3. Diagrama UML de actividades. Elementos clave

- **Nodos** de **Inicio** y **Fin**
  - Inicio: Representado por un círculo negro (●).
  - Fin: Representado por un círculo negro con un contorno blanco (◎).
- **Actividades**
  - Representan una **acción** o **tarea** dentro del proceso.
  - Se dibujan como un. **rectángulo** con **esquinas redondeadas**.
  - Ejemplo: "Validar credenciales", "Procesar pago", "Generar reporte".
- **Transiciones** (Flechas)
  - Indican el **flujo** del proceso de una actividad a otra.
  - Representan la **dirección** de la **ejecución**.

3. Diagrama UML de actividades. Elementos clave

- **Decisiones** y **Condiciones**
  - Representadas por **rombos**.
  - Definen bifurcaciones en el flujo según condiciones.
  - Se etiquetan con condiciones, como "Pago aprobado", "Pago rechazado".
- **Bifurcación** y **Unión**
  - Se utilizan para modelar **procesos concurrentes**.
  - **Fork (Bifurcación):** Representado por una **línea gruesa horizontal o vertical**, divide un flujo en múltiples actividades concurrentes.
  - **Join (Unión):** Une múltiples flujos en una única secuencia.
- **Objetos** y **Flujos** de **Datos**
  - Representan datos o información **que se mueve entre actividades**.
  - Se dibujan como un **rectángulo con una etiqueta**.
  - Se conectan con una **flecha punteada**.

3. Diagrama UML de actividades. Elementos clave

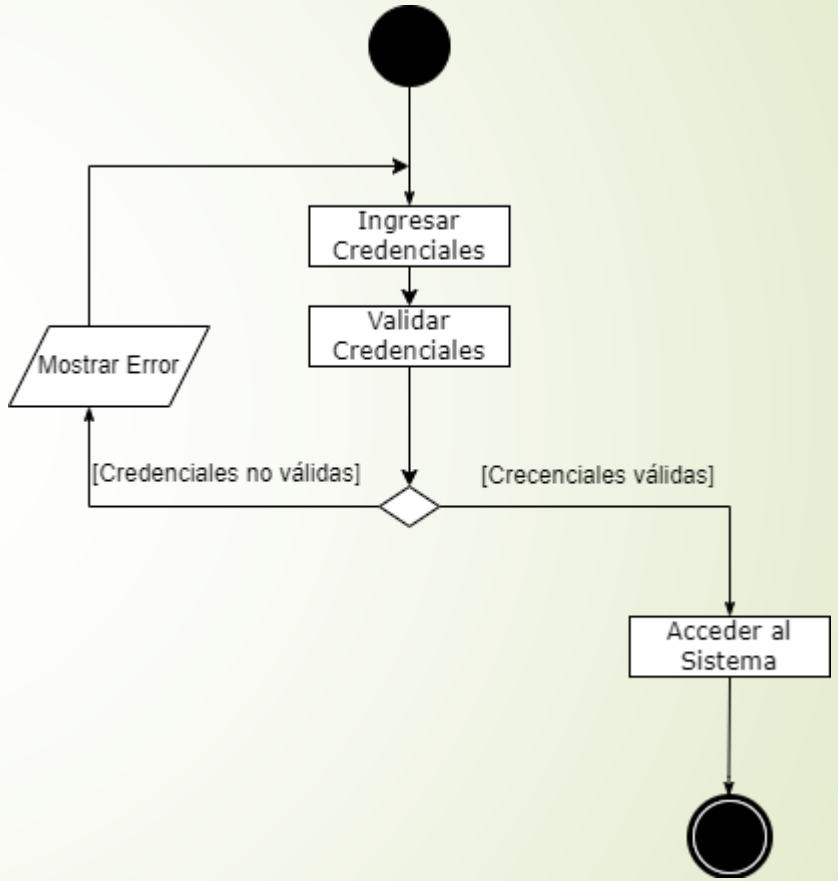
- ¿Para qué se **usan**?
  - Modelar el **flujo de trabajo** de un proceso de negocio.
  - Describir el **comportamiento** de **un caso de uso**.
  - Representar **algoritmos** y **lógica condicional**.
  - Modelar **procesos paralelos** y **sincronización** en sistemas concurrentes.
  - Definir **flujos de trabajo** en sistemas software o empresariales.

3. Diagrama UML de actividades. Elementos clave

Ejemplo: Proceso de Login

- El usuario ingresa credenciales.
- El sistema valida las credenciales.
  - Si son correctas, accede al sistema.
  - Si son incorrectas, muestra un error y permite intentarlo nuevamente.
- Fin del proceso.

3. Diagrama UML de actividades. Ejemplo Práctico



Proceso de Login. Diagrama:

4. Diagrama UML de Secuencia

- La relación entre los **casos de uso** y los **diagramas de secuencia** en UML es estrecha y complementaria, ya que ambos modelan aspectos del comportamiento del sistema, pero desde diferentes perspectivas y niveles de detalle.
- **Casos de uso**: Se centran en una visión macro, describiendo las funcionalidades principales que el sistema debe ofrecer desde la perspectiva de los usuarios (actores). Muestran **qué** debe hacer el sistema, **no cómo** lo hace.
- Diagramas de **secuencia**: Se enfocan en el nivel micro, detallando **cómo** se realiza **un caso de uso** específico mediante **interacciones** entre objetos o componentes del sistema.
- Los **casos de uso** son el punto de partida para identificar los **requisitos** del sistema.
- Basándose en ellos, se crean los diagramas de **secuencia** para **detallar cómo** los **actores interactúan** con los **objetos internos** del sistema para cumplir con la funcionalidad descrita en el caso de uso.
- Los **casos de uso** establecen **qué funcionalidades** se esperan del sistema, mientras que los diagramas de **secuencia** detallan **cómo esas funcionalidades** se implementan e interactúan en términos de **mensajes** e **interacciones** entre objetos. Trabajan juntos para ofrecer una comprensión clara de los requisitos y la dinámica del sistema.

4. Diagrama UML de secuencia

- Implementar un diagrama de **secuencia** implica representar las **interacciones** dinámicas entre los actores y los objetos del sistema en un contexto específico, normalmente vinculado a un caso de uso. Los pasos clave para implementarlo son:
- **A. Identificar el propósito del diagrama**
  - Define el escenario específico que quieres modelar (por ejemplo, el flujo principal o una alternativa de un caso de uso).
  - Pregúntate: ¿Qué interacción quiero representar y con qué nivel de detalle?

4. Diagrama UML de secuencia

- **B. Identificar los elementos clave**
  - **Actores**: Son las entidades externas que interactúan con el sistema (usuarios, otros sistemas, dispositivos, etc.).
    - Identifícalos desde el caso de uso o los requisitos del sistema.
  - **Objetos del sistema**: Identifica los componentes o clases internas del sistema que participan en el escenario.
    - Usa nombres que sean representativos (por ejemplo, CarritoDeCompras, BaseDeDatos).
  - **Mensajes**: Determina las acciones que se realizan (por ejemplo, solicitudes, respuestas, validaciones).
    - Cada mensaje debe expresar la interacción en el flujo del escenario.

4. Diagrama UML de secuencia

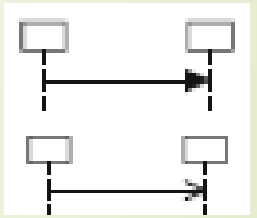
- **C. Crear las líneas de vida**
  - Representa cada actor y objeto como una **línea de vida** (vertical) que muestra su existencia durante la interacción.
  - Dibuja los actores y objetos como rectángulos horizontales en la parte superior del diagrama, con líneas punteadas verticales que bajan desde ellos.

4. Diagrama UML de secuencia

- **D. Dibujar los mensajes**
  - Secuencia lógica:
    - Coloca los mensajes en el orden cronológico en que ocurren.
    - Usa flechas horizontales entre las líneas de vida para mostrar mensajes enviados (flechas **sólidas**) y respuestas (flechas **punteadas** opcionales).



  - Tipos de mensajes:
    - **Síncronos**: El emisor espera una respuesta antes de continuar.
    - **Asíncronos**: El emisor continúa sin esperar una respuesta.



  - Nombres claros: Etiqueta cada mensaje con el nombre del método o acción, y opcionalmente agrega parámetros (por ejemplo, validarUsuario(usuario, contraseña)).

4. Diagrama UML de secuencia

- **E. Agregar activaciones**
  - Muestra períodos de tiempo en los que un objeto o actor está activo (procesando una operación) con un rectángulo angosto en la línea de vida.
  - Indica claramente qué mensajes inician o terminan activaciones.
- **F. Considerar condiciones y repeticiones**
  - Usa elementos de control de flujo:
    - **Condiciones**: Representa decisiones con texto entre corchetes (por ejemplo, [Pago válido]).
    - **Bucle** o **repetición**: Utiliza un marco (llamado loop) para representar iteraciones (por ejemplo, while, for).
- **G. Agregar notas (opcional)**
  - Usa **notas** o **anotaciones** para explicar detalles adicionales, como excepciones o referencias a reglas de negocio.
- **H. Revisar el diagrama**
  - Verifica que el diagrama esté completo y que represente todas las interacciones importantes del escenario.
  - Asegúrate de que sea claro y fácil de entender.

4. Diagrama UML de secuencia: Representación de objetos y línea de vida

- En un diagrama de secuencia, los objetos y sus líneas de vida son elementos esenciales para modelar las interacciones dinámicas en un sistema. A continuación, vemos cómo se representan y utilizan:
  - **1. Representación de objetos**

    Los objetos son las entidades que participan en la interacción. Estos pueden ser actores externos (usuarios, sistemas) o elementos internos del sistema (clases, componentes).

    - **Símbolo**: Se representan como un rectángulo con el nombre del objeto.
      - Formato: NombreObjeto:TipoClase
        - Ejemplo: usuario:Cliente (el objeto usuario es una instancia de la clase Cliente).
      - Si el tipo del objeto no es relevante, se puede omitir el tipo y usar solo el nombre (por ejemplo, usuario).
      - Si el objeto es anónimo o no está definido, se usa :TipoClase.
    - **Ubicación**: Se colocan en la parte superior del diagrama, alineados horizontalmente.

4. Diagrama UML de secuencia: Representación de objetos y línea de vida

- **2. Línea de vida**

  La línea de vida representa la existencia del objeto durante la interacción.

  - **Símbolo**: Una línea punteada vertical que se extiende hacia abajo desde el rectángulo del objeto.
  - **Significado**: Muestra el período de tiempo en el que el objeto está involucrado en la interacción. Su longitud depende de la duración de la interacción.

- **3. Activación en la línea de vida**

  Cuando un objeto está procesando una operación, se representa mediante un rectángulo angosto (llamado **barra de activación**) sobre la línea de vida.

  - **Símbolo:** Un rectángulo vertical sobre la línea de vida.
  - **Significado:** Indica que el objeto está activo, ejecutando una operación o esperando un resultado.

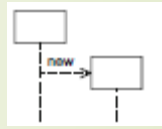
4. Diagrama UML de secuencia: Representación de objetos y línea de vida

- **4. Creación y destrucción de objetos**

  Un mensaje para crear objetos es un tipo especial de mensaje.

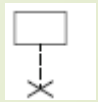
  Creación:

  - Se muestra con una flecha hacia un objeto nuevo.
  - El objeto nuevo aparece en el diagrama con su línea de vida iniciando en el punto de la flecha.
  - Ejemplo: new() es una etiqueta común para métodos de creación.



  Destrucción:

  - La línea de vida termina con una "X".
  - Indica que el objeto deja de existir al final de la interacción.



4. Diagrama UML de secuencia: Representación de objetos y línea de vida

5. Relaciones entre objetos

- Los objetos interactúan mediante mensajes. Los mensajes se representan con flechas horizontales que van de un objeto a otro, mostrando el flujo de comunicación.
- Mensajes síncronos: Línea sólida con punta de flecha completa. Representan llamadas de método donde el remitente espera una respuesta.
- Mensajes asíncronos: Línea sólida con punta de flecha abierta. Representan llamadas donde el remitente no espera una respuesta inmediata.
- Respuestas: Línea punteada con punta de flecha completa que vuelve al remitente.

4. Diagrama UML de secuencia: Ejemplo práctico

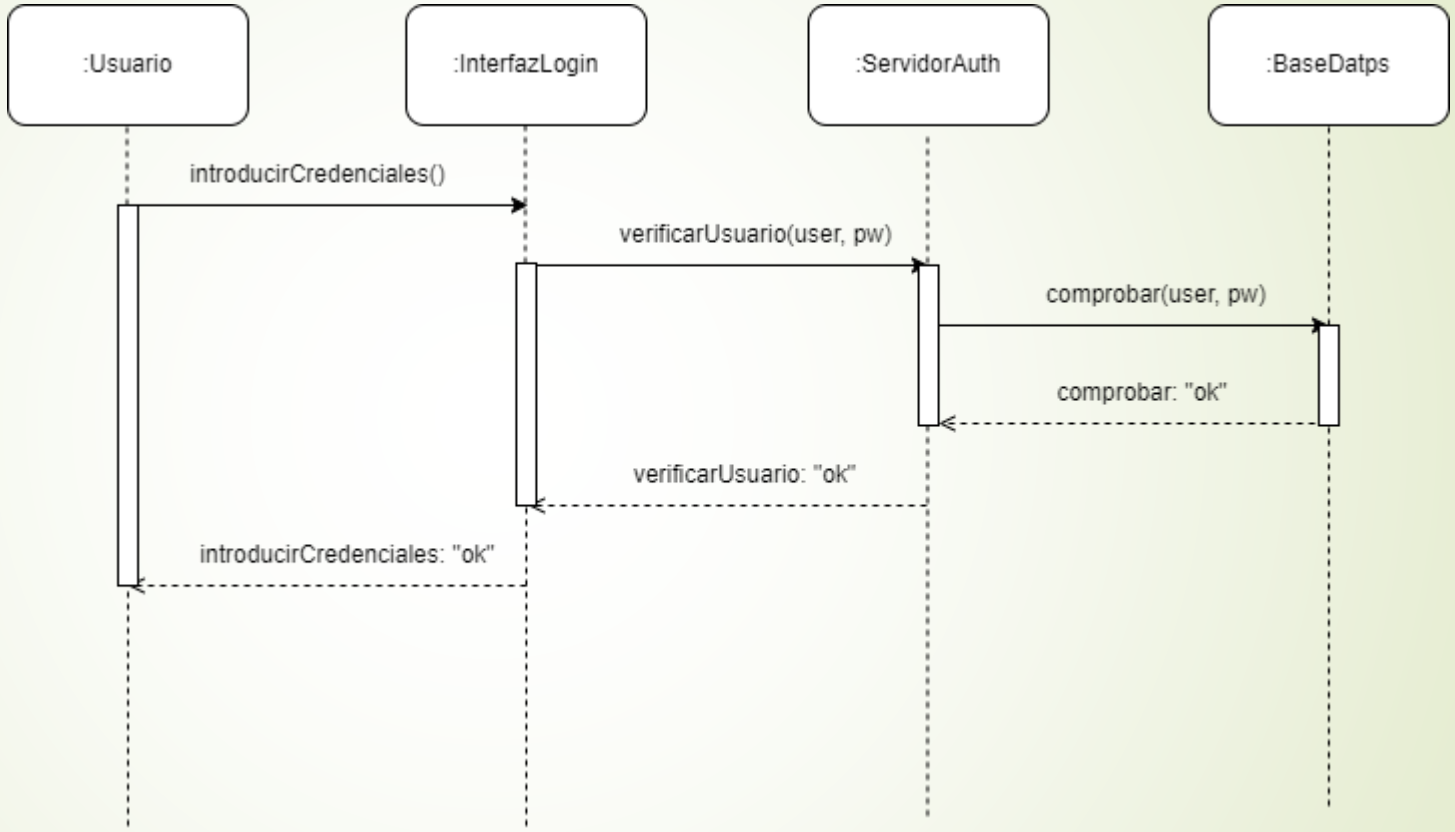
Usuario interactúa con un **sistema de login**:

- **Objetos**: usuario, interfazLogin, servidorAuth, baseDatos.
- **Líneas de vida**: Cada objeto tiene una línea de vida punteada que se extiende desde su rectángulo.
- **Activación**: Cuando el servidor procesa la autenticación, se muestra un rectángulo de activación en su línea de vida.
- **Creación**: Si servidorAuth crea una conexión temporal, su línea de vida comienza en ese punto.
- **Mensajes**:
  - usuario → interfazLogin: introducirCredenciales()
  - interfazLogin → servidorAuth: verificarUsuario()

4. Diagrama UML de secuencia:

Login Process

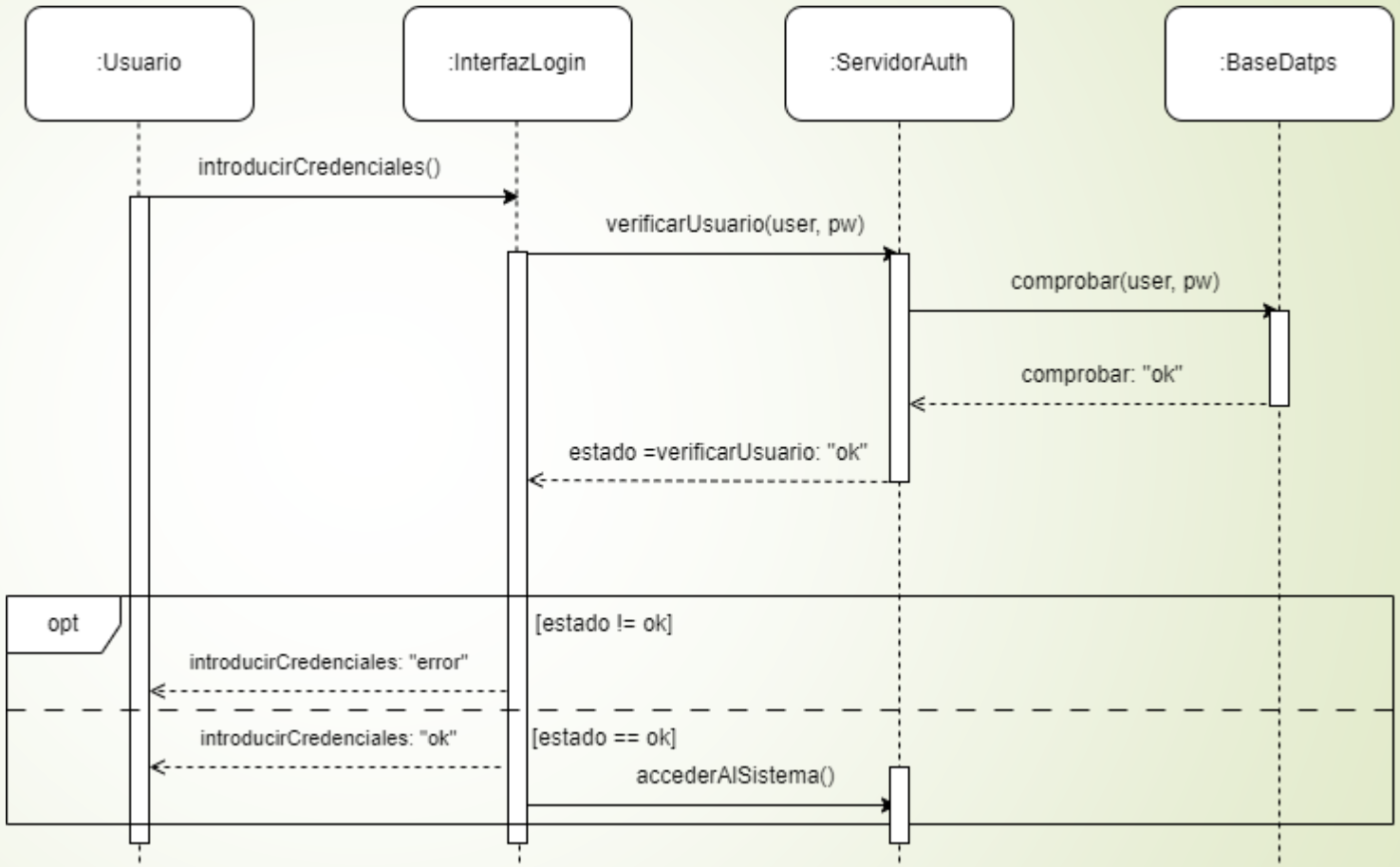
El valor de retorno puede, de forma opcional, ser asignado a una variable. Un mensaje puede etiquetarse con var=m1:valor, donde var es la variable a la que se asigna el valor de retorno, m1 especifica el nombre del mensaje y valor representa el valor de retorno actual.



4. Diagrama UML de secuencia:

Login Process

(versión 2)



4. Diagrama UML de secuencia: Representación de objetos y línea de vida

Buenas prácticas:

- Representa solo los objetos relevantes para el caso que estás modelando.
- Usa nombres claros y consistentes para los objetos y sus métodos.
- Mantén el diagrama organizado, con objetos bien distribuidos horizontalmente para facilitar la lectura.

5. Diagrama UML de comunicación/colaboración

- El diagrama de comunicación contiene la misma información que un diagrama de secuencia, pero la notación es diferente. Presenta diferencias en su enfoque, estructura y utilidad.

Enfoque

DIAGRAMA DE SECUENCIA	DIAGRAMA DE COMUNICACIÓN
Se centra en el **tiempo**	Se centra en **relaciones** estructurales entre objetos
Muestra **orden cronológico** de **mensajes** entre objetos	Representa **cómo** están **organizados** los **objetos** y cómo se **comunican**
Destaca **secuencia exacta** en que ocurren las **interacciones**	Enfatiza la **colaboración** entre elementos para lograr un **propósito**

5. Diagrama UML de comunicación/colaboración

Estructura

DIAGRAMA DE SECUENCIA	DIAGRAMA DE COMUNICACIÓN
Objetos se representan en parte superior con **líneas de vida** (verticales punteadas) que muestran su existencia durante el tiempo	**Objetos** se representan como **círculos** o **rectángulos**, organizados espacialmente en el diagrama
Mensajes se dibujan como flechas horizontales entre las líneas de vida y su posición en el eje vertical indica el orden temporal	Las **líneas** que conectan los objetos, muestran las **relaciones** entre ellos
Puede incluir **activaciones** (rectángulos) para mostrar el tiempo que un objeto está procesando una operación	Los **mensajes** están **numerados** (por ejemplo: 1, 2, 3) para indicar el orden de las interacciones

5. Diagrama UML de comunicación/colaboración

Utilidad (información que transmiten)

DIAGRAMA DE SECUENCIA	DIAGRAMA DE COMUNICACIÓN
Detalla el flujo temporal de los mensajes	Muestra **qué objetos están relacionados** y cómo se comunican
Es útil para analizar y validar el **orden de ejecución** de los eventos	Es útil para analizar **la estructura del sistema** y las relaciones entre sus componentes
Representa explícitamente la **duración** de **activaciones** y el **tiempo de vida** de los **objetos**	Se enfoca más en la **organización espacial** que en el tiempo

5. Diagrama UML de comunicación/colaboración

Utilidad (Usos comunes)

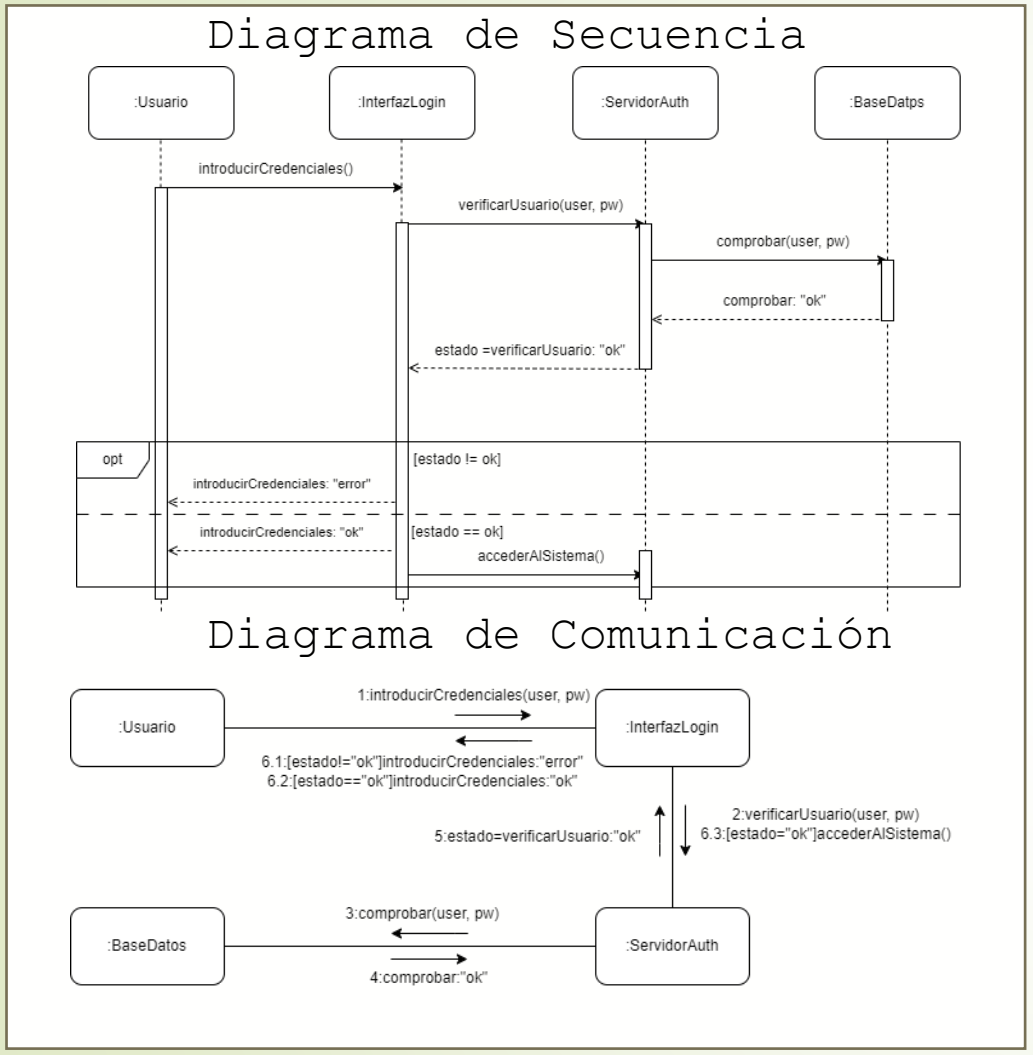
DIAGRAMA DE SECUENCIA	DIAGRAMA DE COMUNICACIÓN
Modelar **escenarios dinámicos** donde el **orden** de los **eventos** es **crítico**, como flujos de trabajo o procesos de negocio	Comprender las **relaciones** entre los **objetos** y cómo **colaboran** para cumplir una funcionalidad
Diseñar y **analizar** el **flujo de mensajes** entre componentes del sistema	Diseñar la **arquitectura** de **interacción** entre **componentes** en un nivel más estructural
Caso: **Proceso de Login** Muestra que primero el usuario introduce sus credenciales, luego el sistema valida en el servidor y finalmente responde con un resultado	**Caso**: **Proceso de Login** Muestra cómo el objeto "Usuario" está conectado al "login Interface", que a su vez está vinculado al "Auth Server" y la "Base de Datos", resaltando las relaciones y el flujo de mensajes entre ellos.

5. Diagrama UML de comunicación/colaboración

Conclusión

- Usa un diagrama de secuencia cuando necesites entender o comunicar el orden y el flujo temporal de los eventos
- Usa un diagrama de comunicación cuando el objetivo sea analizar las conexiones y las relaciones entre los objetos participantes en el sistema

Ambos son complementarios y su elección depende del tipo de información que desees destacar



5. Diagrama UML de comunicación/colaboración

– Login Process (version 2)